EPFL - Computer Science

COM-402 Information Security and Privacy

Lecture — December 2, 2025

Author: Jonathan Dutoit

Professor: Mathias Payer

Trusted Computing

Learning Goals

- Understand ***the need*** for trusted computing
- Learn the ***guarantees*** provided by ***trusted computing***
- Understand the threat model
- Know some ***key technologies*** used in trusted computing
- Understand how violating the threat model can break trusted computing
- Understand the basics of ***side-channel attacks***

Can you trust applications?

All modern systems built on abstractions

Software systems like

- Operating systems / Hypervisors
- Compilers
- Runtime/static libraries

Are extremely complex and maintained by untrusted parties (even open source has bugs)

What about Hardware?

Computer is owned or controlled by another party, e.g., in Cloud computing

Hardware is extremely complex and was built by an untrusted party (e.g., Intel, ARM, Apple) and manufacture by a fab (e.g., TSMC)

Other, untrusted applications run on the computer, e.g., cloud computing, smartphone

Better question: why trust a computer?

Can we trust the hardware?

Trusted Hardware

"A piece of hardware can be trusted, if it always behaves in the expected manner for the indented purpose" - Trusted Computing Group

Trusted Computing: Trusted software (component, verified, analysed) running in a secure hardware container. The guarantees we get is that no one can extract secrets.

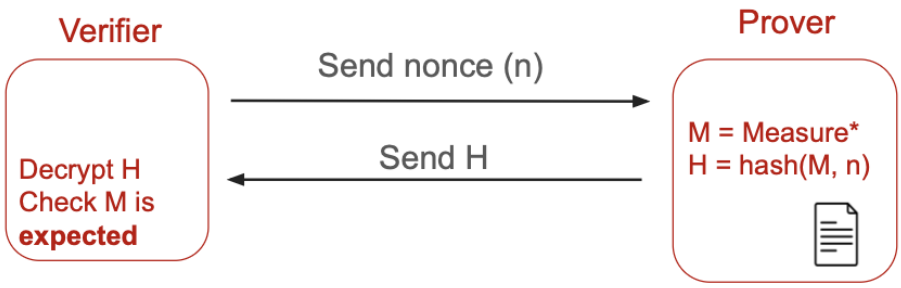
Trusted Hardware Key Properties

- **Attestation**: hardware proves that it does what it's supposed to do
- **Isolation**: privileged code cannot observe computation
- **Sealing**: hardware encrypts data so it can only be accessed when the system meets specific trusted conditions

Attestation

Proves that system is in a specific state to an authorized party.

Attest to a remote authorized party. Is this computer running the correct code?



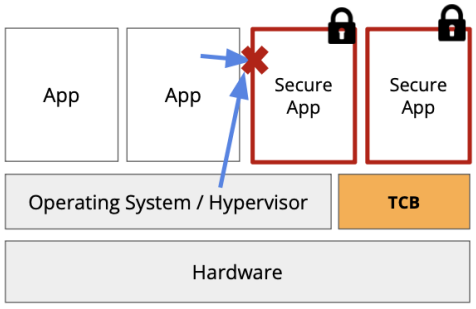
For now measure is magic, ensures safety

Isolation

Constraints who and what can be accessed (access control)

- Unauthorized applications cannot access/write region
- Trusted hardware offers **one well-identified entry-points** to interact with software (to ensure CFI)

The **well-identified** is important to avoid attackers to access any arbitrary access to the application's code (Control Flow Integrity). Imagine a `memcpy` function that could be accessible, hence attackers could use that to copy all fingerprints...



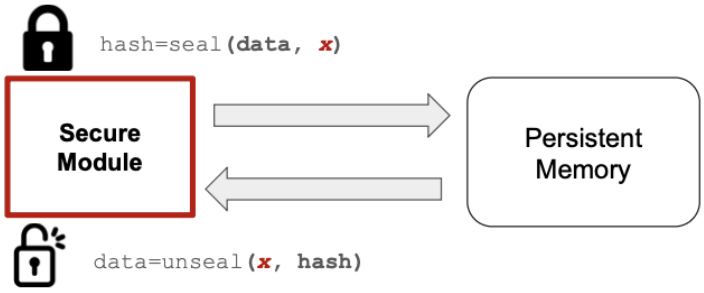
Sealing

Protects private information by binding it to platform configuration or the state.

Store information outside of the trusted environment

X might be the state of the module, or configuration information including the used software and hardware.

Enables secure persistent storage in untrusted memory.



Sealing encrypts data with keys stored inside the TPM, and this data can only be ”unsealed” when the TPM verifies that the current system configuration matches the predefined conditions. This helps protect against unauthorized access or tampering, ensuring the integrity and confidentiality of sensitive data.

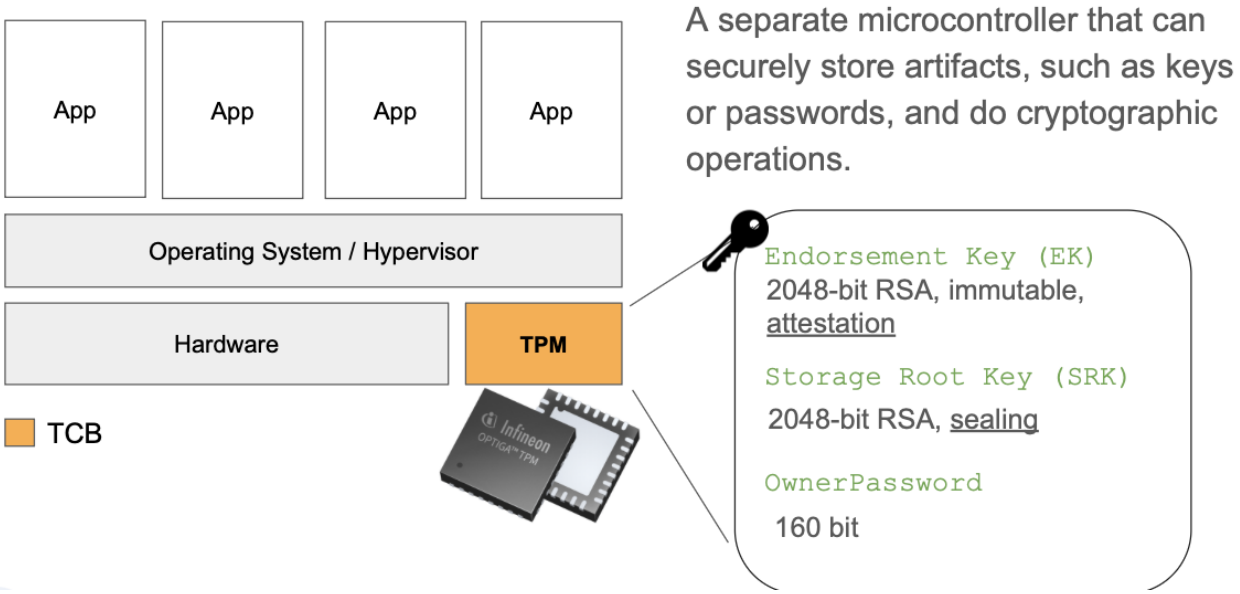
Trusted Hardware

Any hardware that has been certified to perform ***according to a certain set of requirements*** under ***strong adversarial conditions***

- TPM chip
- Hardware Security Module
- Intel SGX

Hardware answers the ”how” trusted computing can be achieved

Trusted Platform Module (TPM)

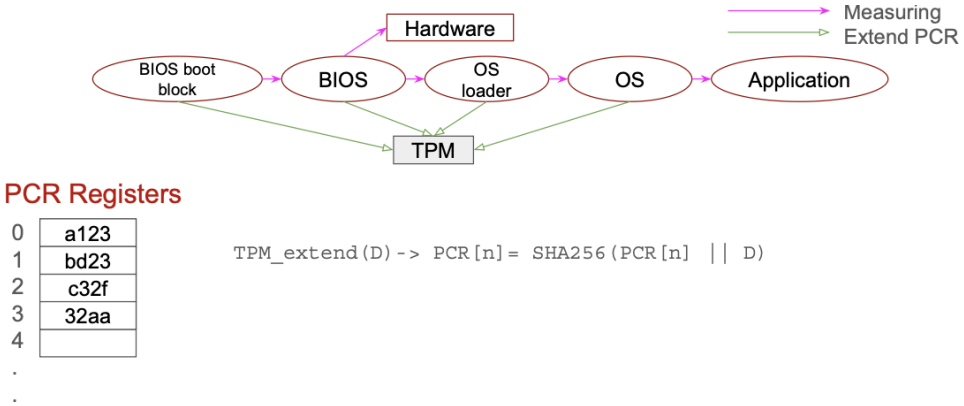


TPM: Attestation

These chips include Platform Configuration Registers (PCR)

- PCRs are initialized at boot time to default value (e.g., 0)
- They are append only
- We can store measurements in these PCRs registers.
- `TPM_extend(D)`: Adds a measurement to TPM
- `TPM_PCRREAD(n)`: returns `PCR[n]`

Example: Secure boot is a perfect example.



In the ***BIOS boot block***: it loads the BIOS in memory, then asks the TPM to a SHA-256 version

of the BIOS (which is a few megabytes) and we compare it to the expected value (which could be hardcoded in hardware, an extension could be if the TPM checks if the BIOS has been ***signed***). If yes, we start the **BIOS**. This is to avoid attacks on the BIOS.

TPM: Sealing

Encrypts a full disk volume

Store: `TPM_Seal(keyhandle, PCRValues, Data)`

Recover: `TPM_Unseal()`

tpm.msc:

- Auto generates 160-bit OwnerPassword
- Stored on TPM and in file `computer_name.tpm`

Volume Master Key (VMK) encrypts disk volume key

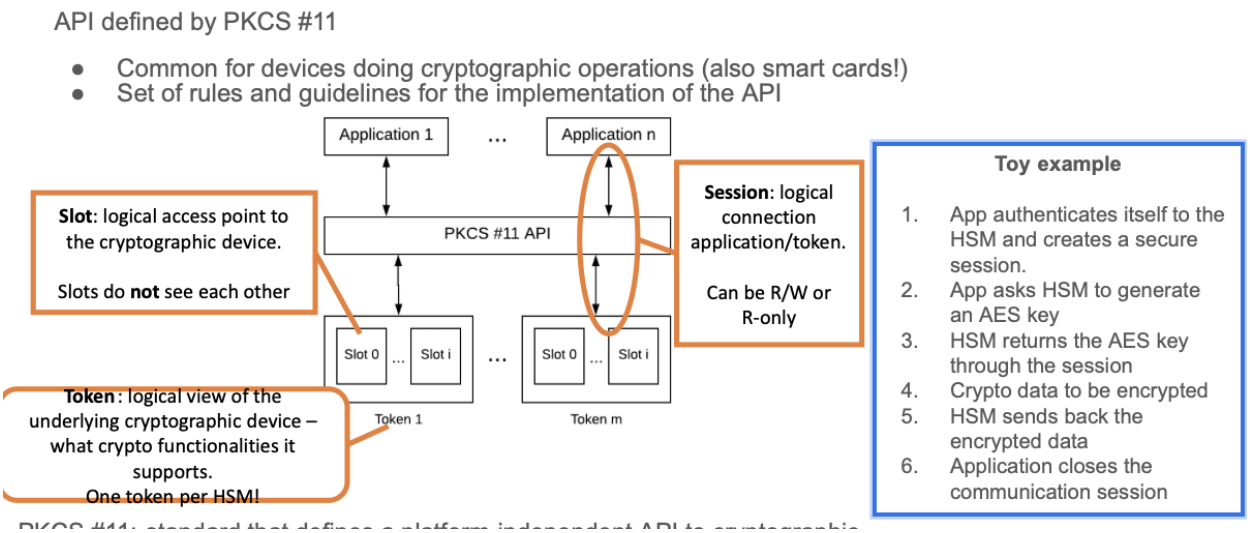
Hardware Secure Modules (HSMs)

External cryptography-oriented device to manage secrets

- generating keys
- encrypting and decrypting data
- and creating and verifying digital signatures

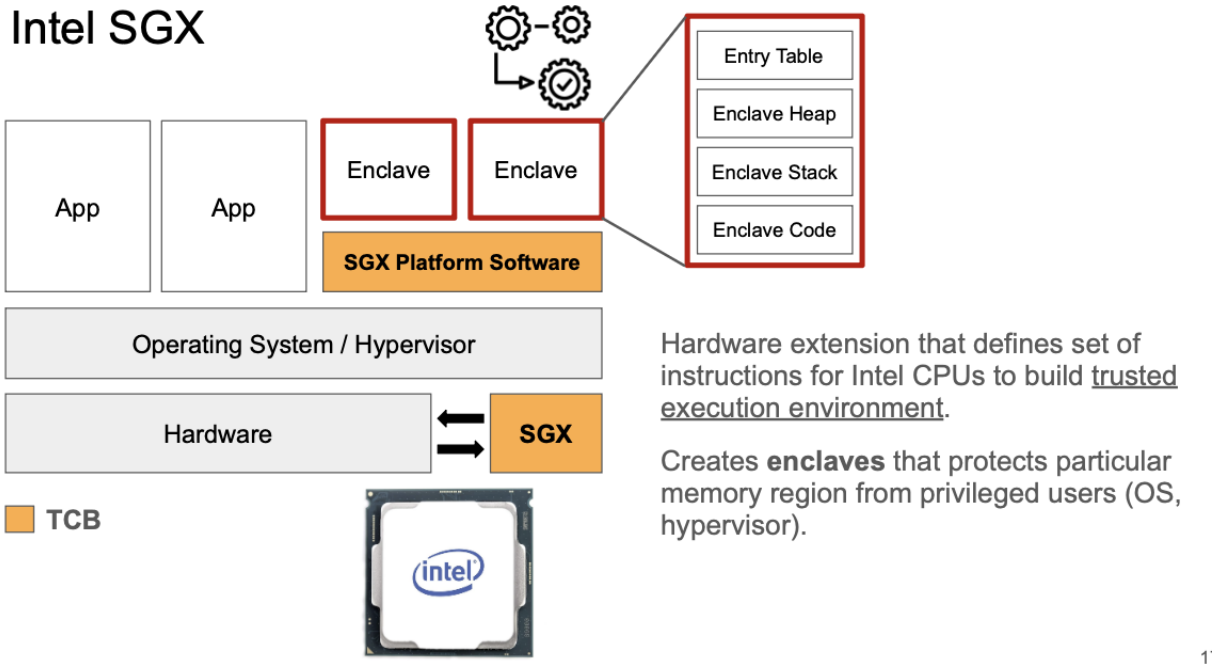
Tamper-resistant hardware device (opening it to steal data won't work)

To flush all data, just delete the key (it's all encrypted with a single key) which is `O(1)` operation. Other alternatives are magnesium, magnets, ...



PKCS 11: standard that defines a platform-independent API to cryptographic tokens, such as hardware security modules (HSM) and smart cards.

Intel SGX



The enclave gives **isolation**.

The enclave cannot call `syscall` functions, it can copy the data already encrypted outside (into the untrusted application)

SGX provides two types of attestation:

- **Local attestation**: an enclave can check other enclave's integrity
- **Remote attestation**: remote party check an enclave's integrity.

Signal

Signal is a messaging application designed to offer strong(er) privacy protection

- End-to-end encryption
- Keys stored on the device
- (Mostly) open source
- Independent, not for profit (funded by donations)

Problem

How to do private contact discovery? In other words, how do find intersection between "Who in my contacts uses Signal" and "Signal Users" database.

Threat model: Who do you want to protect against? Why?
1. Protect from Signal

- **Privacy**: Users do not want to leak any information to Signal about who their contacts are (especially those not using Signal)

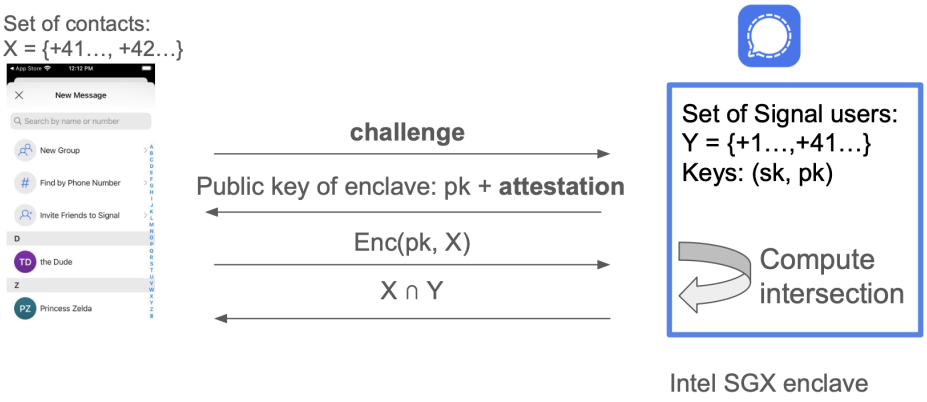
Threat Model: Protect privacy of contacts

- **Access Control?**: No, we cannot... same problem as before, where is the hardware secure module? Signal is a threat in the threat model
- **Cryptography?**: We encrypt the set of contacts in the message and decrypts in the signal application. Oh wait, The idea was **not** to show the contacts signal.
- **Better Cryptography**? Signal's set is very big. PSI does not scale. (PSI = Private Set Intersection)
- **Intel SGX**: SGX secure enclave creates an isolated memory region. Cryptographic keys from Signal. Computer intersection with PSI.

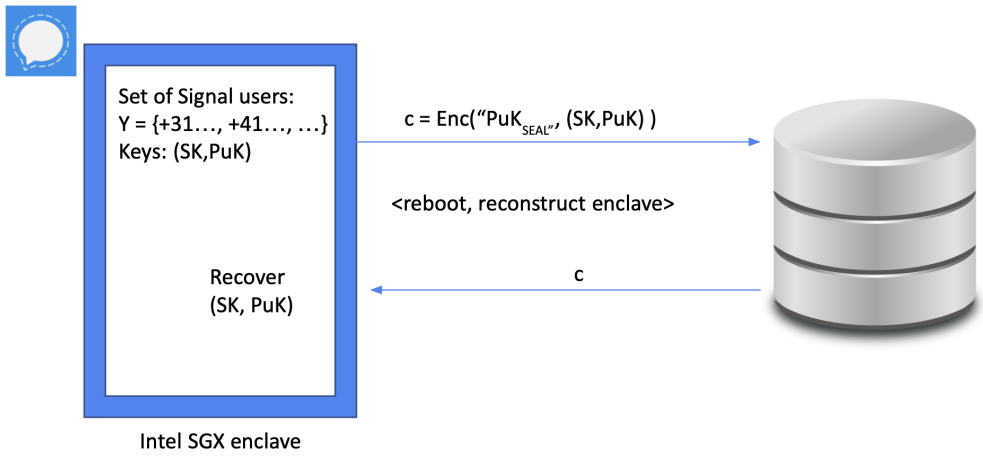
  Challenge: how does the phone know the enclave runs the right program?

  The enclave is open source and the user (phone) can compute (and check) the hash used in attestation

- With **Better** Intel SGX:



Signal: Using Sealing to Store Keys



iPhone Backups

Problem statement: we want users to be able to backup their data to the cloud; the user should be the **only one** that can recover their data

Threat model

Who do you want to protect against/why?

- Other users
- Cloud provider
- Apple
- External parties

Adversaries have lots of computational power!

What about government issues?

Problem: Offer cloud backups for data but guarantee that users retain *exclusive* access

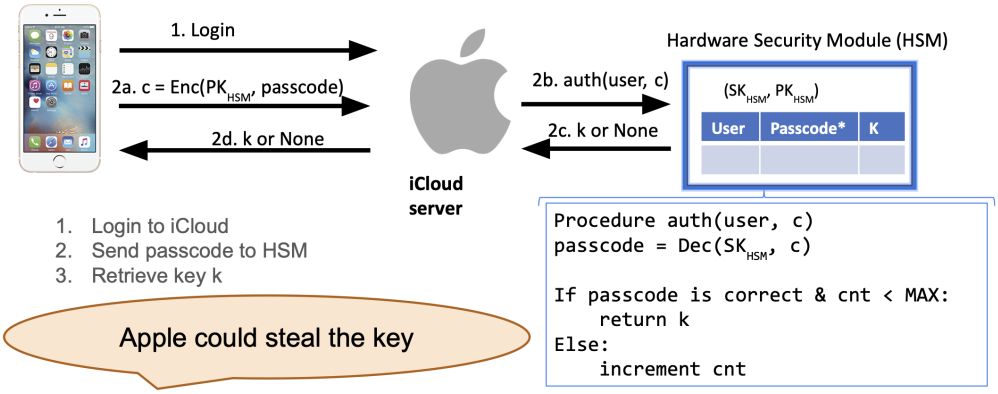
Threat Model: Other users and cloud provider as well as Apple and external parties

- **Access Control**:
  - Problem: ***Location of the Hardware Security Module (HSM)***: Apple and the Cloud are in your threat model!
  - Problem: ***Access control for data***: Even if Apple and the Cloud *were trusted* for running the system, access control just prevents other users and third parties from accessing the data. (And this does not apply to third parties with subpoenas.)
- **Cryptography**: what if you lose the phone?
  - How to decrypt data once the phone is lost, given that users can only remember a short passcode

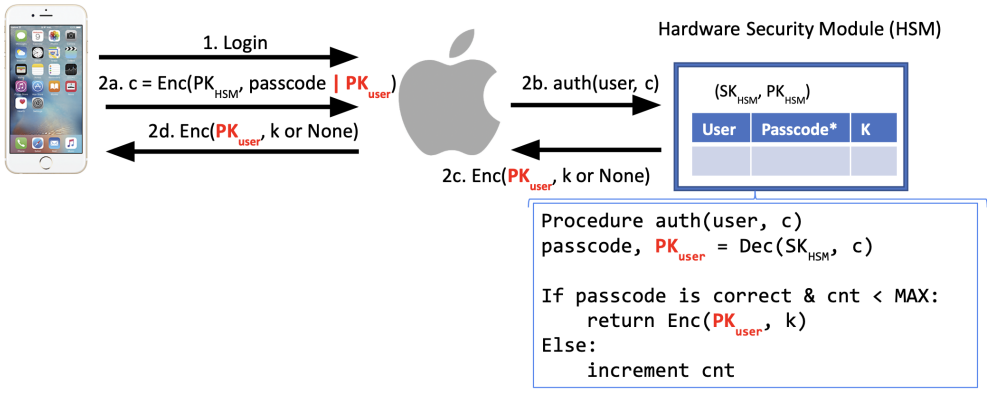
  - How to limit password to only a few attempt? We do not have a HSM that we can check!
  - Good cryptography is made so that if you do not have the key you cannot decrypt the data. A backup system based on a key stored in the phone is pretty useless...

- **Bootstrap cryptography**
  - Problem: ***retrieve data without phone***
    - Cannot rely on "what you have"
  - Problem: ***guessing passwords***
    - Humans are rarely able to remember more than eight characters (and not random!)
  - Problem: ***losing your phone***
- **Storing Passwords**: PK_{HSM} is hardcoded into the device
  - 1. Generate random symmetric key k
  - 2. Encrypt phone data using k, and send data to the cloud
  - 3. Send key k to the HSM, encrypted with PK_{HSM}

Apple's solution



Apple's Secure Backup Solution: Retrieving Key



Apple's Secure Backup: Protecting Software

Apple's HSMs run custom software. This software can be update

How to protect this software?

1. Protect access keys: Apple's HSMs require smartcards to update, stored in sealed and tamper-evident bags while not used
2. Install software
3. Physically destroy access cards

Still need to trust Apple to install secure software in the first place

Smartcards

Plastic card with embedded integrated circuits which can process data and are often tamper resistant. Two main types:

- Memory cards
- Microprocessor cards

Contain keys and certificates: Authentication / digital signatures, confidentiality, key management protocols

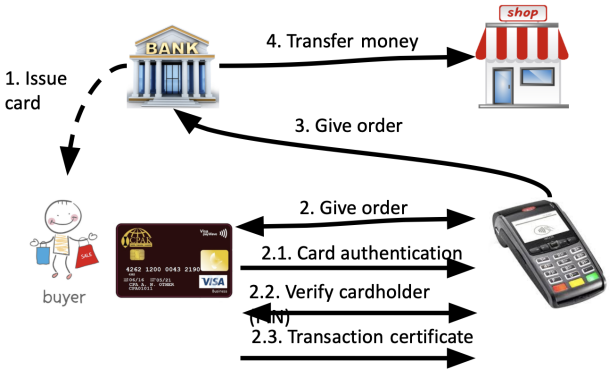
Use-case: Offline Payments

Problem statement: we want users to be able to signal to the bank to pay to a merchant without talking to the bank.

Threat Model:

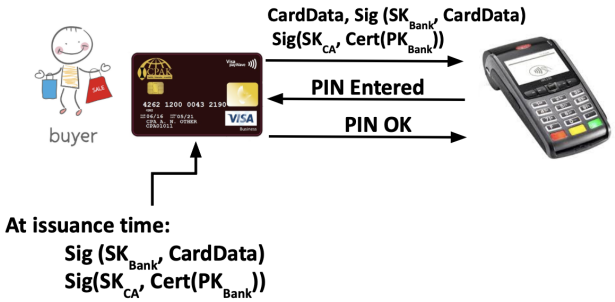
- Other users
- Merchant
- User herself
- External parties

Adversaries may have a lot of computational power and time!!



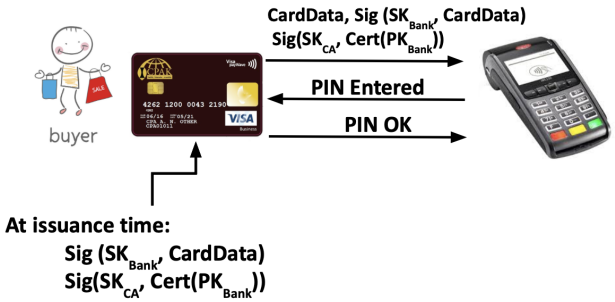
Static Data Authentication

Solution 1: Static Data Authentication
Problem: ***Information can be copied***
No freshness ⇒ Replay is possible!
PIN verification unauthenticated, accept any PIN (YesCard)
Problem: ***Signature can be forged!***



Dynamic Data Authentication

Solution 2: Dynamic Data Authentication
Problem: ***PIN verification unauthenticated***: just accept any PIN (YesCard)
Solution Online checking solves these problems because data will not match, but it is an expensive process, with a risk of money loss



Side channels

Nightmare for trusted hardware

Running a program is a **real** thing

- It takes **time**
- It takes **power**
- **Electromagnetic Emission**

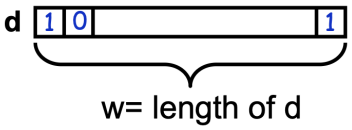
Can we exploit **unintentional leakages** from hardware or software to learn secrets?

Timing Attacks

RSA decryption: compute c^d mod N. A modular exponentiation operation

Naive algorithm: square and multiply

```
s_0 = 1
for k = 0 to w-1 do
    if d[k] == 1 then
        R_k = (s · c) mod n
    else
        R_k = s_k
    end if
    s_{k+1} = R_k^2 mod n
end for
Return R_{w-1}
```



Whether iteration takes a long time depends on the k_th bit of secret exponent.

Computing $(c \cdot s)$ *mod* n takes a while but $R_k = s_k$ is instantaneous

Timing depends on number of 1's

Power Analysis

Elliptic-curve cryptography operation. Power depends on key bits. This could be analysed with a Oscilloscope.

Electromagnetic Emissions

Key Extraction from Mobile Devices via Non intrusive Physical Side Channels

Countermeasures Against Side Channels

Goal: Prevent secret inference from observable state

Hiding: lowers signal to noise ration

Noise generator, randomized execution order, asynchronous logic styles. ...

Masking: (secret sharing) splits state into shares; forces adversary to recombine

Boolean or arithmetic masking, Higher-order masking

Leakage Resilience: prevents leakage aggregation by updating secret

Micro-architectural Side Channels

Meltdown

Catastrophic attack making it possible to read all memory of a process (including **protected kernel data**)

By-product of the way some micro-architectural features are implemented (e.g., AMD x86 implementations are **per chance** resistant to Meltdown)

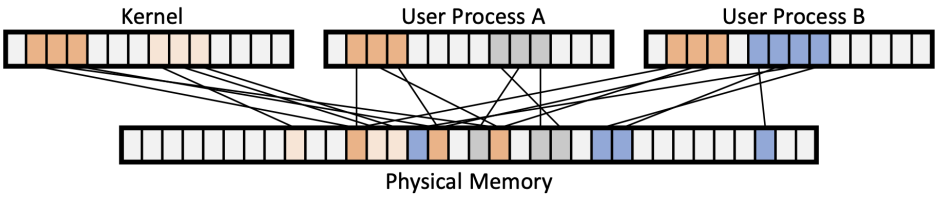
Exploits **race condition** between **memory access** and **protection checks**

Ultimately exploits the micro-architectural nature of caches (something is left in the cache upon exception because **the cache is not part of the architectural state**

> The attacker executes a **forbidden access** and **speculatively uses the result** to obtain **non-architectural side-effects** that reveal the secrets **before the forbidden access is squashed**.

Mapping Kernel Pages in User Space

Most OSes mapped physical kernel memory pages into every user's virtual memory space



Minimizes the cost of some exceptions (e.g., fast interrupt handling, less TLB flushes)

Of course, access is protected, i.e., data can be read only in kernel mode

But everyone can address them!

Processors	Affected?
Intel x86	Most processors since 1995
AMD x86	None
ARM	Cortex-A75
Apple ARM	Most processors
IBM POWER	POWER8 and POWER9
VIA x86	Most processors

Possible Mitigations

The obvious proper solutions it to **change the processor design**

- Test privilege level **before** making the result of a speculative access available
- By chance, AMD does this already

The other line of mitigation is to **isolate user space and kernel** space memory

- In Linux, **Kernel page-table isolation (KPTI)**: similar in other OSes
- Performance penalty in Linux around 5-10%, up to 30%

Spectre

Another catastrophic attack making it possible to **read all memory**.

Addresses another shared resource: **branch predictors**

- For simplicity, brand predictors are not unique per-thread (but shared among SMT threads)

Exploits side effects of **(miss-predicted) speculative execution**

- Mispeculation does not affect the architectural state (of course!)...
- ...but it may **affect micro-architectural structures** (e.g., caches)

> Get the victim to **speculatively** execute **leaky** code whose **non-architectural side-effects** reveal the secrets

Possible Mitigations

Hardware

- Disable speculative execution
- Separate branch predictors per process/thread

General software approaches

- Run only an application per processor

Partial and application-specific software approaches

- Add serialization instructions between branches and loads
- Make it impossible through JavaScript in browsers

Not easy to fix!

Even With Trusted Hardware, Vendor Requires *Some*

Trust

Backup data system from Apple

HSM manufacturer: controls the production of the ”black box”

Apple: Must be trusted to install the correct code the first time (attestation helps)

Discovery of private contacts for Signal (based on Intel SGX)

Intel: Controls the production of the (black box) SGX system. It also controls the attestation keys built into every device.

Signal: Must be trusted to not leak any data from the mobile application but not for running the correct set intersection code (the attestation allows to check that)

Trusted Hardware: Lessons Learned

It exits, and offers three properties:

- **Isolation**: it is not possible to ”peek” inside
- **Attestation**: it can prove that it does what you think it is doing
- **Sealing**: it can store secrets in memory that can only be recovered by itself

Having trusted hardware doesn't mean you don't need to think about protocols!

Building trusted hardware is difficult: side channel attacks

Trusted hardware means to trust the manufacturer!

Separation of privilege: Use several cards with advanced cryptography.